

ΠΡΟΧΩΡΗΜΕΝΑ ΘΕΜΑΤΑ
ΤΕΧΝΟΛΟΓΙΑΣ ΚΑΙ ΕΦΑΡΜΟΓΩΝ
ΒΑΣΕΩΝ ΔΕΔΟΜΕΝΩΝ

ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΕΡΓΑΣΙΑ ΓΙΑ
ΤΟ ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ 2025-2026

ΟΜΑΔΑ

ΑΠΟΣΤΟΛΟΣ ΜΑΡΜΑΓΚΑΣ, 5030

ΒΑΣΙΛΕΙΟΣ ΓΚΕΛΗΣ, 5152

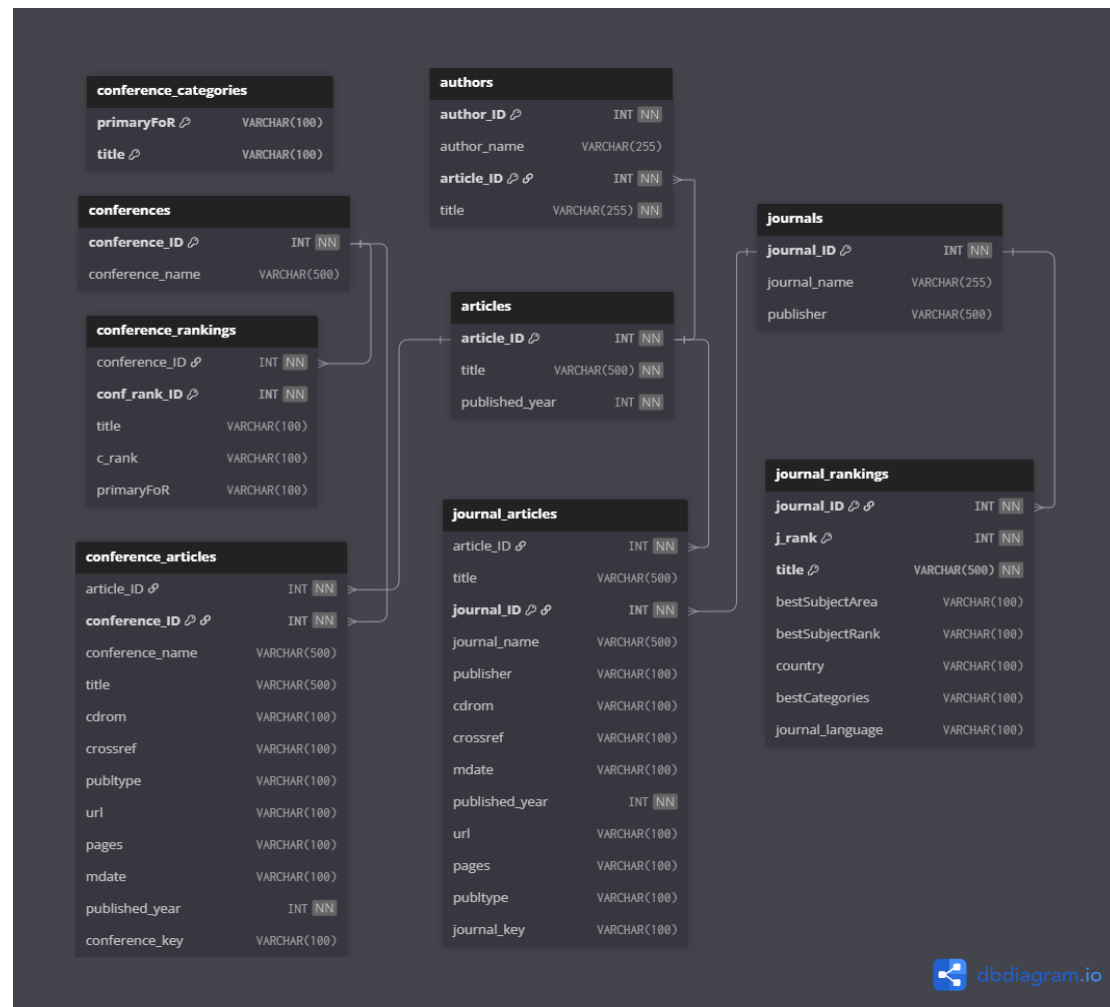
ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

ΜΑΪΟΣ 2026

1 ΒΑΣΗ ΔΕΔΟΜΕΝΩΝ

Στην παρούσα ενότητα περιγράφονται τα σχήματα της βάσης δεδομένων που χρησιμοποιούνται στο project.

1.1 ΣΧΕΣΙΑΚΟ ΣΧΗΜΑ ΣΕ ΛΟΓΙΚΟ ΕΠΙΠΕΔΟ



Σχήμα 1.1 Σχεσιακό σχήμα της βάσης δεδομένων του συστήματος

Section-break (continuous)

```
-- MySQL dump 10.13 Distrib 9.6.0, for Win64 (x86_64)
--
-- Host: localhost Database: mye30_db
--
-- Server version 9.6.0

/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!50503 SET NAMES utf8mb4 */;
/*!40103 SET @OLD_TIME_ZONE=@@TIME_ZONE */;
/*!40103 SET TIME_ZONE='+00:00' */;
/*!40014 SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0 */;
```

```
/*!40014 SET @@OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0 */;
/*!40101 SET @@OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='NO_AUTO_VALUE_ON_ZERO' */;
/*!40111 SET @@OLD_SQL_NOTES=@@SQL_NOTES, SQL_NOTES=0 */;
SET @MYSQLDUMP_TEMP_LOG_BIN = @@SESSION.SQL_LOG_BIN;
SET @@SESSION.SQL_LOG_BIN= 0;

--
-- GTID state at the beginning of the backup
--

SET @@GLOBAL.GTID_PURGED=/*!80000 '+'*/ 'a742c5b9-1a35-11f1-83ec-94de80c809eb:1-62401';

--
-- Current Database: `mye30_db`
--

CREATE DATABASE /*!32312 IF NOT EXISTS*/ `mye30_db` /*!40100 DEFAULT CHARACTER SET utf8mb4
COLLATE utf8mb4_0900_ai_ci */ /*!80016 DEFAULT ENCRYPTION='N' */;

USE `mye30_db`;

--
-- Table structure for table `articles`
--

DROP TABLE IF EXISTS `articles`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `articles` (
  `article_ID` int NOT NULL,
  `title` varchar(500) NOT NULL,
  `published_year` int NOT NULL,
  PRIMARY KEY (`article_ID`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
/*!40101 SET character_set_client = @saved_cs_client */;

--
-- Dumping data for table `articles`
--

LOCK TABLES `articles` WRITE;
/*!40000 ALTER TABLE `articles` DISABLE KEYS */;
/*!40000 ALTER TABLE `articles` ENABLE KEYS */;
UNLOCK TABLES;

--
-- Table structure for table `authors`
--

DROP TABLE IF EXISTS `authors`;
/*!40101 SET @saved_cs_client      = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `authors` (
  `author_ID` int NOT NULL,
  `author_name` varchar(255) NOT NULL,
  `article_ID` int NOT NULL,
  `title` varchar(255) NOT NULL,
  PRIMARY KEY (`author_ID`,`author_name`,`article_ID`),
  KEY `article_ID` (`article_ID`),
  CONSTRAINT `authors_ibfk_1` FOREIGN KEY (`article_ID`) REFERENCES `articles` (`article_ID`)
ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
/*!40101 SET character_set_client = @saved_cs_client */;

--
-- Dumping data for table `authors`
--
```

1.2 ΣΧΕΣΙΑΚΟ ΣΧΗΜΑ ΣΕ ΦΥΣΙΚΟ ΕΠΙΠΕΔΟ

Εδώ καταγράφονται οι ρυθμίσεις σε φυσικό επίπεδο. Ενδεικτικά: Δημιουργήσαμε μια βάση δεδομένων στο MySQL Workbench, με τις προτεινόμενες (default) ρυθμίσεις.

1.2.1 ΡΥΘΜΙΣΗ ΤΩΝ ΠΑΡΑΜΕΤΡΩΝ ΤΟΥ DBMS

Variable_name	Value
default_storage_engine	InnoDB
innodb_buffer_pool_size	134217728
innodb_log_buffer_size	16777216
join_buffer_size	262144
key_buffer_size	8388608
soin_buffer_size	262144

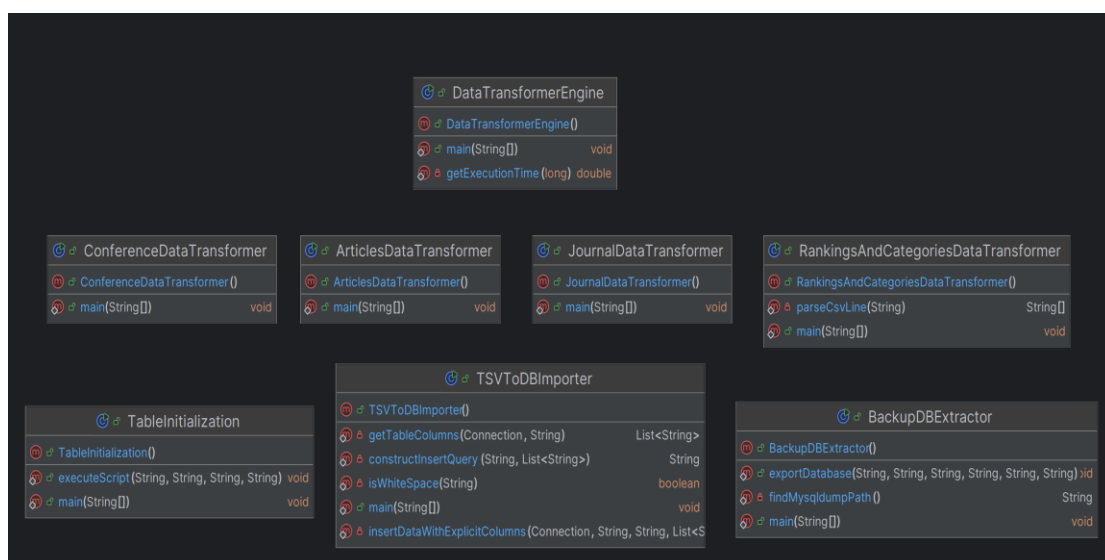
2 ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΛΟΓΙΣΜΙΚΟΥ

2.1 ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΚΑΙ ΔΟΜΗ ETL

Μέσω του etl πακέτου επεξεργαζόμαστε τα δεδομένα στη βάση μας, ώστε να δημιουργηθούν τα transformed αρχεία που θα εισάγουμε στο τέλος.

Αρχικά, στα JournalDataTransformer και ConferenceDataTransformer εξάγουμε την πληροφορία που θέλουμε για τους πίνακες journals, conferences και δημιουργούμε κάποια temporary αρχεία, τα οποία χρησιμοποιούνται στο υπόλοιπο πακέτο. Μέσω αυτών των αρχείων παίρνουμε την πληροφορία για τον ArticleDataTranformer, που δημιουργεί την συγκεντρωτική πληροφορία για τα άρθρα, αλλά και τα τελικά αρχεία για περιοδικά, συνέδρια και συγγραφείς. Αντίστοιχη διαδικασία υλοποιείται για τα rankings μέσω του RankingsAndCategoriesDataTransformer, όπου αντιστοιχίζεται η πληροφορία με βάση τον τίτλο του περιοδικού/συνεδρίου. Η εκτέλεση των παραπάνω προγραμμάτων καλείται μέσω του DataTransformedEngine.

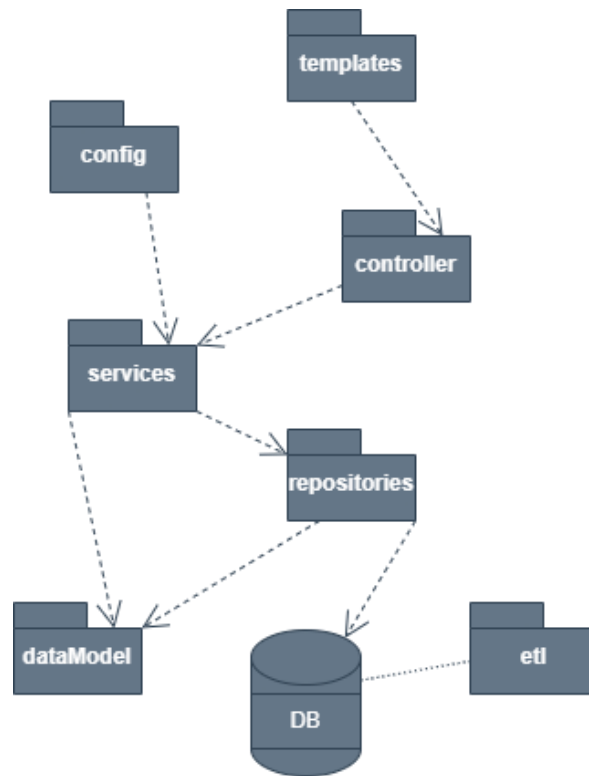
Δημιουργούμε τους πίνακες της βάσης με το TableInitialization και έπειτα μέσω του TSVToDBImporter εισάγουμε όλα τα δεδομένα των επεξεργασμένων αρχείων στους πίνακες. Τέλος, το BackupDBExtractor δημιουργεί ένα backup των δεδομένων της βάσης μας.



Σχήμα 2.1 Κλάσεις πακέτου 'etl'.

2.2 ΔΙΑΓΡΑΜΜΑΤΑ ΠΑΚΕΤΩΝ / ΥΠΟΣΥΣΤΗΜΑΤΩΝ ΚΕΝΤΡΙΚΗΣ ΕΦΑΡΜΟΓΗΣ

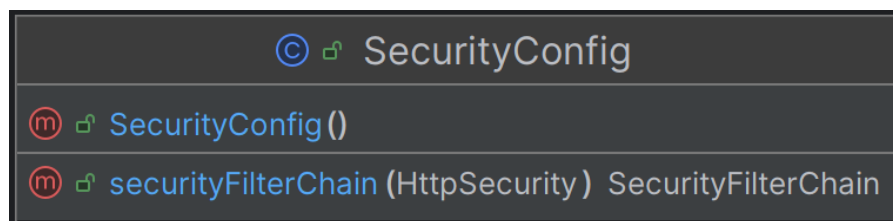
Το διάγραμμα για τα πακέτα του λογισμικού που κατασκευάσατε ως κεντρική εφαρμογή επερώτησης. Ο στόχος είναι να φανεί η high-level αρχιτεκτονική του συστήματος, χωρίς λεπτομέρειες των επί μέρους κλάσεων.



Σχήμα 2.2 Packages Diagram

2.3 ΔΙΑΓΡΑΜΜΑ(ΤΑ) ΚΛΑΣΕΩΝ ΚΕΝΤΡΙΚΗΣ ΕΦΑΡΜΟΓΗΣ

Εδώ παρουσιάζονται τα διαγράμματα κλάσεων της εφαρμογής, εκτός του πακέτου etl που παρουσιάζεται στην προηγούμενη ενότητα.



Εικόνα 1 Κλάσεις πακέτου 'config'.

MainController		
main	MainController ()	
main	printJournalStatsRange (String, int, int, Model)	void
main	printConferenceStatsRange (String, int, int, Model)	void
main	getListConferences (Model)	String
main	getGroupedBarCharts (String, String, String, Model)	String
main	printConferenceRanking (String, Model)	void
main	getAuthorProfile (String, Model)	String
main	extractArticleCountsByCategory (List<String>, List<String>, List<String>)	void
main	getJournalProfile (String, Integer, Integer, Model)	String
main	printJournalStats (String, Model)	void
main	getHomeMenu (Model)	String
main	printJournalRanking (String, Model)	void
main	getLineCharts (List<String>, List<String>, List<String>, List<String>)	void
main	getListJournals (Model)	String
main	extractArticleCounts (List<String>, List<String>, List<String>, List<String>)	void
main	getListAuthors (Model, int)	String
main	getListYears (Model)	String
main	extractAuthorCounts (List<String>, List<String>, int, int, List<String>)	void
main	printConferenceStats (String, Model)	void
main	getYearProfile (Integer, String, String, String, Model)	String
main	getConferenceProfile (String, Integer, Integer, Model)	String

```

graph TD
    subgraph AuthorService
        AS1[findNumArticlesByYear String List<Object[]>]
        AS2[findLastPublishedYear String Integer]
        AS3[findAllAuthors Page<String> Page<String>]
        AS4[findAvgArticlesByYear String Double]
        AS5[findFirstPublishedYear String Integer]
        AS6[findAllArticles String List<Article>]
    end

    subgraph JournalService
        JS1[findJournalByFirstYear String Integer]
        JS2[findAvgAuthorsByYear String Double]
        JS3[findNumOfJournalsByCategory String List<Object[]>]
        JS4[findAllJournals List<String>]
        JS5[findNumOfJournalsAuthors String Integer]
        JS6[findJournalsArticlesRange String, int, int List<Author>]
        JS7[findAvgJournalsArticlesRange String, int, int Double]
        JS8[findJournalByLastYear String Integer]
        JS9[findJournalsArticles String List<Article>]
        JS10[findJournalsAuthors String List<Author>]
        JS11[findAvgAuthorsNumArticlesByYear String List<Object[]>]
        JS12[findJournalsArticlesRange String, int, int List<Article>]
        JS13[findNumOfJournalsArticlesByYear String List<Object[]>]
        JS14[findAvgJournalsArticles String Double]
        JS15[findPublisherPublications String Integer]
        JS16[findJournalsRanking String JournalRanking]
        JS17[findAvgAuthorsByJournal String Double]
        JS18[findNumOfJournalsArticlesByYear String List<Object[]>]
        JS19[findAvgAuthorsByJournalRange String, int, int Double]
        JS20[findAvgAuthorsByYearRange String, int, int Double]
        JS21[findNumOfJournalsAuthorsRange String, int, int Integer]
    end

    subgraph ConferenceService
        CS1[findNumOfConferenceArticlesByYear String List<Object[]>]
        CS2[findAvgAuthorsByYearRange String, int, int Double]
        CS3[findConferenceArticlesRange String, int, int List<Article>]
        CS4[findConferenceArticles String List<Article>]
        CS5[findAvgAuthorsByConferenceRange String, int, int Double]
        CS6[findConferenceRanking String ConferenceRanking]
        CS7[findAllConferences List<String>]
        CS8[findAvgConferenceArticles String Double]
        CS9[findAvgConferenceArticlesRange String, int, int Double]
        CS10[findNumOfConferenceAuthorsRange String, int, int Integer]
        CS11[findConferenceAuthors String List<Author>]
        CS12[findConferencesByLastYear String Integer]
        CS13[findAvgAuthorsByConference String Double]
        CS14[findAvgAuthorsByYear String Double]
        CS15[findNumOfConferenceAuthorsByYear String List<Object[]>]
        CS16[findNumOfConferenceAuthors String Integer]
        CS17[findConferencesByFirstYear String Integer]
        CS18[findAvgAuthorsNumArticlesByYear String List<Object[]>]
        CS19[findConferenceAuthorsRange String, int, int List<Author>]
        CS20[findNumOfConferenceByCategory String List<Object[]>]
    end
  
```

The diagram illustrates three Java service classes, each with a set of methods and their return types. The classes are represented by boxes with a title bar and a list of methods. The methods are listed in a table-like format within each box.

- AuthorService** (Title: **AuthorService**):
 - `findNumArticlesByYear (String) List<Object[]>`
 - `findLastPublishedYear (String) Integer`
 - `findAllAuthors (Page<String>) Page<String>`
 - `findAvgArticlesByYear (String) Double`
 - `findFirstPublishedYear (String) Integer`
 - `findAllArticles (String) List<Article>`
- JournalService** (Title: **JournalService**):
 - `findJournalByFirstYear (String) Integer`
 - `findAvgAuthorsByYear (String) Double`
 - `findNumOfJournalsByCategory (String) List<Object[]>`
 - `findAllJournals () List<String>`
 - `findNumOfJournalsAuthors (String) Integer`
 - `findJournalsArticlesRange (String, int, int) List<Author>`
 - `findAvgJournalsArticlesRange (String, int, int) Double`
 - `findJournalByLastYear (String) Integer`
 - `findJournalsArticles (String) List<Article>`
 - `findJournalsAuthors (String) List<Author>`
 - `findAvgAuthorsNumArticlesByYear (String) List<Object[]>`
 - `findJournalsArticlesRange (String, int, int) List<Article>`
 - `findNumOfJournalsArticlesByYear (String) List<Object[]>`
 - `findAvgJournalsArticles (String) Double`
 - `findPublisherPublications (String) Integer`
 - `findJournalsRanking (String) JournalRanking`
 - `findAvgAuthorsByJournal (String) Double`
 - `findNumOfJournalsArticlesByYear (String) List<Object[]>`
 - `findAvgAuthorsByJournalRange (String, int, int) Double`
 - `findAvgAuthorsByYearRange (String, int, int) Double`
 - `findNumOfJournalsAuthorsRange (String, int, int) Integer`
- ConferenceService** (Title: **ConferenceService**):
 - `findNumOfConferenceArticlesByYear (String) List<Object[]>`
 - `findAvgAuthorsByYearRange (String, int, int) Double`
 - `findConferenceArticlesRange (String, int, int) List<Article>`
 - `findConferenceArticles (String) List<Article>`
 - `findAvgAuthorsByConferenceRange (String, int, int) Double`
 - `findConferenceRanking (String) ConferenceRanking`
 - `findAllConferences () List<String>`
 - `findAvgConferenceArticles (String) Double`
 - `findAvgConferenceArticlesRange (String, int, int) Double`
 - `findNumOfConferenceAuthorsRange (String, int, int) Integer`
 - `findConferenceAuthors (String) List<Author>`
 - `findConferencesByLastYear (String) Integer`
 - `findAvgAuthorsByConference (String) Double`
 - `findAvgAuthorsByYear (String) Double`
 - `findNumOfConferenceAuthorsByYear (String) List<Object[]>`
 - `findNumOfConferenceAuthors (String) Integer`
 - `findConferencesByFirstYear (String) Integer`
 - `findAvgAuthorsNumArticlesByYear (String) List<Object[]>`
 - `findConferenceAuthorsRange (String, int, int) List<Author>`
 - `findNumOfConferenceByCategory (String) List<Object[]>`

Εικόνες 2,3 Κλάσεις πακέτων 'controller' και 'service' (το serviceImp περιέχει ακριβώς τις ίδιες κλάσεις).

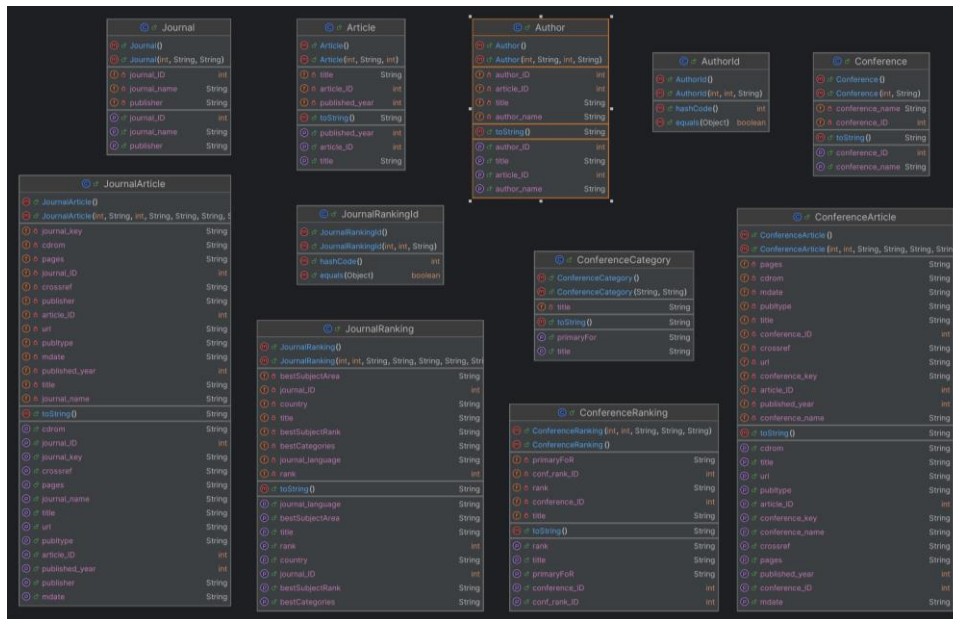
① ↻ YearRepository		
findNumOfConferences (int)	Integer	
findNumOfAuthors (int)	List<Object[]>	
findPublicationByConference (int, String)	List<Object[]>	
findPublicationByAuthor (int, String)	List<Object[]>	
findNumOfJournals (int)	Integer	
findAllYears ()	List<Integer>	
findPublishedArticles (int)	Integer	
findPublicationByJournal (int, String)	List<Object[]>	

① ↻ JournalRepository		
findAvgAuthorsByYearRange (String, int, int)	Double	
findAvgJournalArticles (String)	Double	
findAvgAuthorsByJournalRange (String, int, int)	Double	
findAvgAuthorsByJournal (String)	Double	
findAllJournals ()	List<String>	
findNumOfJournalAuthorsByYear (String)	List<Object[]>	
findAvgAuthorsByYear (String)	Double	
findJournalByLastYear (String)	Integer	
findJournalByFirstYear (String)	Integer	
findJournalAuthorsRange (String, int, int)	List<Author>	
findAvgAuthorsNumArticlesByYear (String)	List<Object[]>	
findNumOfJournalByCategory (String)	List<Object[]>	
findNumOfJournalArticlesByYear (String)	List<Object[]>	
findNumOfJournalAuthors (String)	Integer	
findJournalRanking (String)	JournalRanking	
findNumOfJournalAuthorsRange (String, int, int)	Integer	
findPublisherPublications (String)	Integer	
findJournalAuthors (String)	List<Author>	
findAvgJournalArticlesRange (String, int, int)	Double	
findJournalArticles (String)	List<Article>	
findJournalArticlesRange (String, int, int)	List<Article>	

① ↻ AuthorRepository		
findNumArticlesByYear (String)	List<Object[]>	
findAvgArticlesByYear (String)	Double	
findAllAuthors (Pageable)	Page<String>	
findFirstPublishedYear (String)	Integer	
findLastPublishedYear (String)	Integer	
findAllArticles (String)	List<Article>	

① ↻ ConferenceRepository		
findNumOfConferenceAuthorsRange (String, int, int)	Integer	
findConferenceArticles (String)	List<Article>	
findAvgConferenceArticles (String)	Double	
findConferenceByFirstYear (String)	Integer	
findAvgConferenceArticlesRange (String, int, int)	Double	
findAllConferences ()	List<String>	
findAvgAuthorsByYear (String)	Double	
findConferenceArticlesRange (String, int, int)	List<Article>	
findNumOfConferenceAuthorsByYear (String)	List<Object[]>	
findConferenceRanking (String)	ConferenceRanking	
findAvgAuthorsByConference (String)	Double	
findConferenceAuthorsRange (String, int, int)	List<Author>	
findConferenceAuthors (String)	List<Author>	
findAvgAuthorsNumArticlesByYear (String)	List<Object[]>	
findNumOfConferenceArticlesByYear (String)	List<Object[]>	
findNumOfConferenceByCategory (String)	List<Object[]>	
findAvgAuthorsByYearRange (String, int, int)	Double	
findNumOfConferenceAuthors (String)	Integer	
findConferenceByLastYear (String)	Integer	
findAvgAuthorsByConferenceRange (String, int, int)	Double	

Εικόνα 4 Κλάσεις πακέτου 'repositories'.



Εικόνα 5 Κλάσεις πακέτου 'dataModel'.

3 ΥΠΟΔΕΙΓΜΑΤΑ ΕΡΩΤΗΣΕΩΝ ΚΑΙ ΑΠΑΝΤΗΣΕΩΝ

Ένα ενδεικτικό παράδειγμα για τα Journals:

```
public class MainController {
    // Journal

    @GetMapping("/journalProfile")
    public String getJournalProfile(@RequestParam("journalName") String journalName, @RequestParam(value = "minYear", required = false) Integer minYear, @RequestParam(value = "maxYear", required = false) Integer maxYear, Model model) {
        printJournalRanking(journalName, model);
        model.addAttribute("journalName", journalName);
        if (minYear != null && maxYear != null) {
            printJournalStatsRange(journalName, minYear, maxYear, model);
            model.addAttribute("isFiltered", true);
            model.addAttribute("minYear", minYear);
            model.addAttribute("maxYear", maxYear);
        } else {
            printJournalStats(journalName, model);
            model.addAttribute("isFiltered", false);
        }
        return "journalProfile";
    }

    private void printJournalRanking(String name, Model model) {
        if (name != null) {
            String journalName = name.trim();
            JournalRanking ranking = journalService.findJournalRanking(name);
            model.addAttribute("ranking", ranking);
        } else {
            model.addAttribute("ranking", null);
        }
    }

    private void printJournalStats(String name, Model model) {
        Integer firstYear = journalService.findJournalByFirstYear(name);
        Integer lastYear = journalService.findJournalByLastYear(name);
        Integer numofAuthors = journalService.findNumofJournalAuthors(name);
        Double avgAuthorsByJournal = journalService.findAvgAuthorsByJournal(name);
        Double avgAuthorsByYear = journalService.findAvgAuthorsByYear(name);
    }
}
```

```
package my5030project_db3_5030_5152.service;

import java.util.List;

public interface JournalService {

    List<String> findAllJournals();

    JournalRanking findJournalRanking(String journal_name);

    List<Object[]> findNumofJournalArticlesByYear(String journal_name);

    List<Object[]> findNumofJournalAuthorsByYear(String journal_name);

    Integer findJournalByLastYear(String journal_name);

    Integer findJournalByFirstYear(String journal_name);

    Integer findNumofJournalAuthors(String journal_name);

    Double findAvgAuthorsByJournal(String journal_name);

    Double findAvgAuthorsByYear(String journal_name);

    List<Article> findJournalArticles(String journal_name);

    List<Author> findJournalAuthors(String journal_name);

    Double findAvgJournalArticles(String journal_name);

    Integer findNumofJournalAuthorsRange(String journal_name, int y1, int y2);

    Double findAvgAuthorsByJournalRange(String journal_name, int y1, int y2);

    Double findAvgAuthorsByYearRange(String journal_name, int y1, int y2);
}
```

```
@Repository
public interface JournalRepository extends JpaRepository<Journal, Integer> {

    @Query("SELECT j FROM Journal j")
    List<String> findAllJournals();

    @Query("SELECT j FROM Journal j INNER JOIN JournalRanking jr ON j.journal_id=jr.journal_id WHERE j.journal_name=?")
    JournalRanking findJournalRanking(String journal_name);

    @Query("SELECT ja.published_year, COUNT(DISTINCT (art.article_id)) FROM Article art INNER JOIN JournalArticle ja ON art.article_id=ja.article_id WHERE ja.journal_name=? GROUP BY ja.published_year")
    List<Object[]> findNumofJournalArticlesByYear(String journal_name);

    @Query("SELECT ja.published_year, COUNT(ath.author_id), COUNT(DISTINCT (ath.author_id)) FROM Author ath INNER JOIN JournalArticle ja ON ath.article_id=ja.article_id WHERE ja.journal_name=? GROUP BY ja.published_year")
    List<Object[]> findNumofJournalAuthorsByYear(String journal_name);

    // Stats

    @Query("SELECT MAX(ja.published_year) FROM Journal j INNER JOIN JournalArticle ja ON j.journal_id=ja.journal_id WHERE j.journal_name=?")
    Integer findJournalByLastYear(String journal_name);

    @Query("SELECT MIN(ja.published_year) FROM Journal j INNER JOIN JournalArticle ja ON j.journal_id=ja.journal_id WHERE j.journal_name=?")
    Integer findJournalByFirstYear(String journal_name);

    @Query("SELECT COUNT(DISTINCT(ath.author_id)) FROM Author ath INNER JOIN JournalArticle ja ON ath.article_id=ja.article_id WHERE ja.journal_name=?")
    Integer findNumofJournalAuthors(String journal_name);

    @Query("SELECT COUNT(ath.author_id) * 1.0 / COUNT(DISTINCT ja.article_id) FROM Author ath INNER JOIN JournalArticle ja ON ath.article_id=ja.article_id WHERE ja.journal_name=?")
    Double findAvgAuthorsByJournal(String journal_name);

    @Query("SELECT COUNT(ath.author_id) * 1.0 / COUNT(DISTINCT ja.published_year) FROM Author ath INNER JOIN JournalArticle ja ON ath.article_id=ja.article_id WHERE ja.journal_name=?")
    Double findAvgAuthorsByYear(String journal_name);

    @Query("SELECT art FROM Article art INNER JOIN JournalArticle ja ON art.article_id=ja.article_id WHERE ja.journal_name=?")
    List<Article> findJournalArticles(String journal_name);
}
```

4 ΛΟΙΠΑ ΣΧΟΛΙΑ

Αρχικά, να αναφερθεί πως τα path για τα αρχεία των καταλόγων data, transformed_data είναι generic. Παρόλα αυτά υπάρχει περίπτωση να χρειάζεται ακόμα ένα όνομα καταλόγου, το οποίο και θα βρείτε σε σχόλια μέσα στο string του path.

Έπειτα, καλό θα ήταν να γίνει download όλος ο φάκελος MYE30-Project και όχι μόνο το db3_5030_5152.

Τέλος, για την εκτέλεση, και επειδή δεν περιέχονται στο παραδοτέο τα TSV αρχεία του transformed_data, η εκτέλεση της εφαρμογής θα πρέπει να είναι η εξής:

- Βήμα 1: Συνδεόμαστε στο MySQL workbench και εκτελούμε την εντολή CREATE DATABASE mye30_db;
- Βήμα 2: Εκτελούμε τον DataTransformerEngine.java
- Βήμα 3: Εκτελούμε τον TableInitialization.java
- Βήμα 4: Εκτελούμε τον TSVToDBImporter.java
- Βήμα 5: Εκτελούμε τον BackupDBExtractor.java
- Βήμα 6: Εκτελούμε τον Db350305152Application για να δούμε το website στον σύνδεσμο <http://localhost:8080/homeMenu>

Σημείωση: Λόγω του μεγάλου όγκου πληροφορίας που έχει το αρχείο για τους authors, υπάρχει πιθανότητα η εκτέλεση του importer να καθυστερήσει στο σημείο αυτό. Συγκεκριμένα, λόγω του περιορισμένου χώρου στο δίσκο των μηχανημάτων που δοκιμάστηκε ο χρόνος αναμονής ήταν περίπου εβδομήντα λεπτά. Σε αντίστοιχη περίπτωση προτείνεται η διακοπή του, ώστε να ελεγχθεί η εφαρμογή με όσες εγγραφές έχουν ήδη φορτωθεί. Στο βίντεο της παρουσίασης όμως έχει ολοκληρωθεί κανονικά.